TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
ĐỀ TÀI

Giảng viên hướng dẫn: **Đào Thị Lệ Thủy**

Sinh viên thực hiện: **Dương Quang Huy - 241230742**
Trần Quang Khải - 241230756
Hoàng Bá Trung - 241224491

Lớp: **CNTT 2-K65**

Khoa: **Công nghệ thông tin**

Hà Nội - 2026

MỤC LỤC

MỤC LỤC HÌNH ẢNH 4
LỜI MỞ ĐẦU 6
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 7
1.1. Lý do chọn đề tài 7
1.2. Mục tiêu và phạm vi dự án 7
1.3. Công nghệ và công cụ sử dụng 7
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 9
2.1. Mô tả trò chơi và luật chơi 9
2.1.1. Luật chơi cơ bản 9
2.1.2. Thiết lập không gian 9
2.2. Biểu đồ Use Case 9
2.3. Thiết kế luồng hoạt động 10
2.3.1. Luồng xác thực người dùng (Login / Register) 11
2.3.2. Luồng chọn màn chơi 11
2.3.3. Luồng vòng lặp trò chơi (Game Loop) 12
2.4. Thiết kế trạng thái hệ thống 12
2.5. Phân tích kiến trúc tổ chức 12
CHƯƠNG 3: CÀI ĐẶT CHI TIẾT 14
3.1. Lớp Model – Dữ liệu và trạng thái 14
3.1.1. Class Map 14
3.1.2. Class PacMan 15
3.1.3. Class Ghost 16
3.1.4. Class Item 17
3.1.5. Class MenuButton 18
3.1.6. Class GameModel 20
3.2. Lớp Controller – Điều phối logic 22
3.2.1. Vòng lặp game 23
3.2.2. Thuật toán kiểm tra va chạm 24
3.2.3. Di chuyển Pacman 25
3.2.4. Thuật toán AI Ghost 26
3.2.5. Xử lý va chạm PacMan – Ghost và PacMan – Item 27

3.2.6. Điều kiện thắng/thua và chuyển màn 27
3.2.7. Xử lý chuyển cảnh theo GameState 28
3.2.8. InputHandler – Bắt sự kiện bàn phím, chuột, hover 29
3.3. Lớp View – Hiển thị đồ họa 30
3.3.1. GamePanel – Render toàn bộ giao diện theo GameState hiện tại 31
3.3.2. Vẽ bản đồ tile-by-tile từ ma trận ký tự 32
3.3.3. Vẽ các màn hình 32
3.3.4. GameWindow – Khung cửa sổ JFrame 33
3.4. Lớp Utils – Dịch vụ dùng chung 34
3.4.1 Class AssetManager 34
3.4.2. Class SoundManager 35
3.4.3. Class DatabaseManager 36
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ 39
4.1. Kết quả đạt được 39
4.1.1. Demo màn hình Login / Register 39
4.1.2. Demo màn hình chọn màn chơi 40
4.1.3. Demo màn hình Gameplay thực tế 41
4.1.4. Demo màn hình GameOver/GameWon 42
4.2. Đánh giá ưu điểm 43
4.3. Hạn chế và hướng phát triển 44
4.3.1. Hạn chế 44
4.3.2. Hướng phát triển 44
KẾT LUẬN 46

MỤC LỤC HÌNH ẢNH

Hình 2. 1: Mô tả thiết lập của bản đồ, nhân vật, vật thể 9
Hình 2. 2: Biểu đồ use case của chương trình 10
Hình 2. 3: Biểu đồ luồng hoạt động của chương trình 11
Hình 2. 4: Cây thư mục của chương trình 13
Hình 3. 1: Bảng các thuộc tính trong class Map 14
Hình 3. 2: Bảng các phương thức trong class Map 14
Hình 3. 3: Bảng các thuộc tính trong class PacMan 15
Hình 3. 4: Bảng các phương thức trong class PacMan 16
Hình 3. 5: Bảng các thuộc tính trong class Ghost 17
Hình 3. 6: Bảng các phương thức trong class Ghost 17
Hình 3. 7: Bảng các thuộc tính trong class Item 18
Hình 3. 8: Bảng các phương thức trong class Item 18
Hình 3. 9: Bảng các thuộc tính trong class MenuButton 19
Hình 3. 10: Bảng các phương thức trong class MenuButton 20
Hình 3. 11: Bảng các thuộc tính trong class GameModel 21
Hình 3. 12: Bảng các phương thức trong class GameModel 22
Hình 3. 13: Bảng các thuộc tính trong class GameController 23
Hình 3. 14: Bảng các phương thức trong class GameController 24
Hình 3. 15: Bảng các thuộc tính liên quan đến canMove 24
Hình 3. 16: Bảng các phương thức liên quan đến canMove 25
Hình 3. 17: Bảng các thuộc tính liên quan đến queued turn 25
Hình 3. 18: Bảng các phương thức liên quan đến queued turn 26
Hình 3. 19: Bảng các thuộc tính liên quan đến AI Ghost 26
Hình 3. 20: Bảng các phương thức liên quan đến AI Ghost 27
Hình 3. 21: Bảng các phương thức xử lý va chạm 27
Hình 3. 22: Bảng các phương thức xử lý thắng/thua và chuyển màn 28
Hình 3. 23: Bảng các thuộc tính liên quan đến chuyển cảnh 28
Hình 3. 24: Bảng các phương thức liên quan đến chuyển cảnh 29
Hình 3. 25: Bảng các thuộc tính trong class InputHandler 30
Hình 3. 26: Bảng các phương thức trong class InputHandler 30
Hình 3. 27: Bảng các thuộc tính trong class GamePanel 31
Hình 3. 28: Bảng các phương thức trong class GamePanel 31
Hình 3. 29: Bảng các phương thức vẽ bản đồ và nhân vật 32
Hình 3. 30: Bảng các phương thức vẽ các màn hình 32
Hình 3. 31: Bảng các phương thức trong class GamWindow 34
Hình 3. 32: Bảng các thuộc tính trong class AssetManager 34
Hình 3. 33: Bảng các phương thức trong class AssetManager 35
Hình 3. 34: Bảng các thuộc tính trong class SoundManager 35
Hình 3. 35: Bảng các phương thức trong class SoundManager 36

Hình 3. 36: Bảng các thuộc tính trong class DatabaseManager 37
Hình 3. 37: Bảng các phương thức trong class DatabaseManager 37
Hình 4. 1: Màn hình Login 39
Hình 4. 2: Màn hình Register 39
Hình 4. 3: Màn hình chọn level 40
Hình 4. 4: Popup level khóa 41
Hình 4. 5: Popup thông tin level 41
Hình 4. 6: Màn hình Gameplay 42
Hình 4. 7: Màn hình tạm dừng 42
Hình 4. 8: Màn hình GameOver 43
Hình 4. 9: Màn hình GameWon 43

LỜI MỞ ĐẦU

Pac-Man – tựa game arcade kinh điển ra đời năm 1980 – đã trở thành biểu tượng của ngành công nghiệp trò chơi điện tử. Chính sự quen thuộc đó đã khơi nguồn ý tưởng cho nhóm: thay vì xây dựng một ứng dụng thuần túy học thuật, tại sao không tái hiện lại một sản phẩm thực sự có thể chơi được? Đó là lý do nhóm em quyết định chọn đề tài: Phát triển lại tựa game kinh điển Pac-Man bằng Java cho bài tập lớn học kỳ này.

Mặc dù Pac-Man có gameplay khá cơ bản, nhưng quá trình xây dựng chương trình lại đòi hỏi việc xử lý logic chặt chẽ. Nhóm em phải giải quyết nhiều bài toán thực tế như: vẽ bản đồ dạng lưới, xử lý va chạm, thiết lập thuật toán di chuyển cho ma (ghost), và duy trì vòng lặp game ổn định ở mức 60 FPS.

Trong project lần này, ứng dụng được viết hoàn toàn bằng ngôn ngữ Java, sử dụng thư viện Swing/AWT cho phần giao diện. Để hệ thống dễ bảo trì, nhóm đã triển khai source code theo mô hình kiến trúc MVC. Đồng thời, game cũng được tích hợp với Database MySQL để xử lý tính năng đăng nhập, đăng ký và lưu trữ điểm cao của từng màn chơi.

Báo cáo dưới đây sẽ trình bày chi tiết quá trình phân tích, thiết kế kiến trúc và những kết quả nhóm đã đạt được. Dù chắc chắn còn những điểm chưa hoàn thiện, nhóm em trân trọng mọi nhận xét từ cô để tiếp tục cải thiện kỹ năng lập trình trong các học kỳ tới.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Lý do chọn đề tài

Đối với sinh viên ngành Công nghệ Thông tin, việc chuyển tiếp từ các bài tập lập trình giao diện Console (chữ trắng nền đen) sang lập trình ứng dụng có giao diện đồ họa (GUI) là một cột mốc quan trọng. Pacman vốn là một tựa game kinh điển, quen thuộc với nhiều thế hệ, nhưng đằng sau lối chơi tưởng chừng đơn giản ấy lại là một bài toán rất thú vị về tư duy logic.

Nhóm em quyết định chọn tựa game này làm dự án môn học vì nó đáp ứng được nhiều tiêu chí học tập thiết thực. Thứ nhất, nó đòi hỏi sự nắm bắt vững vàng về Lập trình hướng đối tượng (OOP) để quản lý nhiều thực thể trên màn hình. Thứ hai, việc tính toán tọa độ, xử lý va chạm (collision detection) và điều hướng nhân vật giúp củng cố tư duy thuật toán. Cuối cùng, việc tự tay clone (làm lại) tựa game này là cơ hội tuyệt vời để thực hành ghép nối một ứng dụng hoàn chỉnh: từ luồng logic bên dưới, giao diện đồ họa bên trên, cho đến việc kết nối với cơ sở dữ liệu để lưu trữ thông tin.

1.2. Mục tiêu và phạm vi dự án

Mục tiêu cốt lõi của dự án là xây dựng thành công một phiên bản Pacman hoạt động mượt mà trên nền tảng Desktop. Game phải đảm bảo các tính năng cơ bản: nhân vật di chuyển trơn tru, ăn vật phẩm tính điểm, ma (ghost) biết tuần tra và hệ thống xử lý va chạm chính xác. Bên cạnh đó, để nâng cao tính thực tế, dự án đặt mục tiêu tích hợp hệ thống tài khoản người dùng, cho phép đăng ký, đăng nhập và lưu trữ tự động tiến trình chơi (điểm số cao nhất, các màn chơi đã mở khóa) vào cơ sở dữ liệu.

Về phạm vi, chương trình cung cấp gameplay cốt lõi: người chơi điều khiển nhân vật di chuyển trong hệ thống lưới (grid) để thu thập vật phẩm và ghi điểm. Hệ thống sẽ tự động tính toán và xử lý các tình huống va chạm (collision) giữa nhân vật, tường chắn và kẻ thù. Các bóng ma (Ghost) trong game được thiết lập logic di chuyển liên tục và tự động rẽ hướng ngẫu nhiên khi đi đến các ngã tư.

Ngoài ra, ứng dụng còn bao gồm một hệ thống quản lý tài khoản cơ bản, cho phép người dùng đăng ký và đăng nhập (Login/Register). Khi trải nghiệm, toàn bộ tiến trình của người chơi như điểm số cao nhất (Highscore) và các màn chơi (Level) đã mở khóa đều được hệ thống ghi nhận và lưu trữ trực tiếp vào cơ sở dữ liệu (Database).

1.3. Công nghệ và công cụ sử dụng

Để hoàn thiện dự án, nhóm đã lựa chọn một bộ công cụ bám sát với chương trình học trên trường, tập trung vào hệ sinh thái Java:

- Java Swing & AWT: Được sử dụng làm nền tảng chính để xây dựng giao diện đồ họa (GUI). Thư viện Graphics của AWT cho phép vẽ trực tiếp các hình ảnh lên màn hình theo từng khung hình.

- Java Sound API: Đóng vai trò xử lý luồng âm thanh. Công cụ này giúp game có thể phát song song nhạc nền (Background Music) lặp lại liên tục và các hiệu ứng âm thanh tức thời (Sound Effects) như tiếng ăn điểm, tiếng click chuột hay âm thanh chiến thắng.

- MySQL & JDBC: Hệ quản trị cơ sở dữ liệu quan hệ MySQL được chọn để lưu trữ thông tin người dùng nhờ tính ổn định và dễ truy vấn. Thư viện JDBC (Java Database Connectivity) đóng vai trò làm cầu nối, giúp mã nguồn Java có thể gửi các câu lệnh SQL trực tiếp xuống Database.

- XAMPP: Môi trường máy chủ ảo cục bộ (Localhost) giúp khởi chạy và quản lý database MySQL một cách nhanh chóng, tiện lợi trong quá trình phát triển và test lỗi.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Mô tả trò chơi và luật chơi

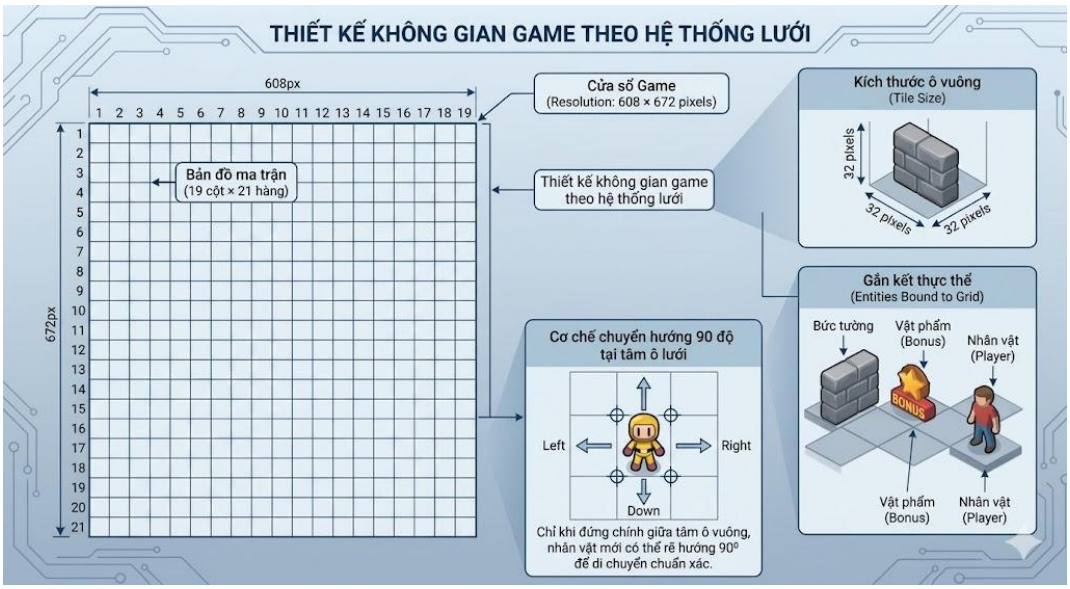
2.1.1. Luật chơi cơ bản

Gameplay của dự án xoay quanh ba yếu tố chính: di chuyển, thu thập và sinh tồn. Người chơi sẽ điều khiển nhân vật Pacman (thông qua các phím WASD hoặc phím mũi tên) đi quanh một mê cung phức tạp. Mục tiêu là ăn hết các hạt Cherry (mỗi hạt trị giá 10 điểm) xuất hiện rải rác trên bản đồ.

Cùng lúc đó, 4 con ma (Red, Pink, Orange, Blue Ghost) sẽ liên tục di chuyển trong mê cung để cản đường. Nếu Pacman chạm phải bất kỳ con ma nào, trò chơi sẽ lập tức kết thúc. Khi toàn bộ Cherry trên bản đồ được thu thập hết, người chơi sẽ giành chiến thắng. Trạng thái chiến thắng này sẽ được hệ thống ghi nhận để mở khóa cấp độ tiếp theo, đồng thời cập nhật điểm số của người chơi vào cơ sở dữ liệu.

2.1.2. Thiết lập không gian

Để dễ dàng quản lý vị trí và tính toán va chạm, không gian trò chơi được thiết kế theo dạng hệ thống lưới. Tổng thể bản đồ là một ma trận gồm 19 cột và 21 hàng. Với kích thước chuẩn của mỗi ô vuông là TILE_SIZE = 32 pixels, cửa sổ game được cố định ở độ phân giải 608 × 672 pixels. Mọi thực thể từ bức tường, vật phẩm đến nhân vật đều được gắn chặt với hệ quy chiếu này. Cơ chế di chuyển cũng được thiết kế chặt chẽ: nhân vật chỉ được phép chuyển hướng 90 độ khi đang đứng chính giữa tâm của một ô lưới, giúp thao tác điều khiển mang lại cảm giác mượt mà và chuẩn xác như tựa game gốc.



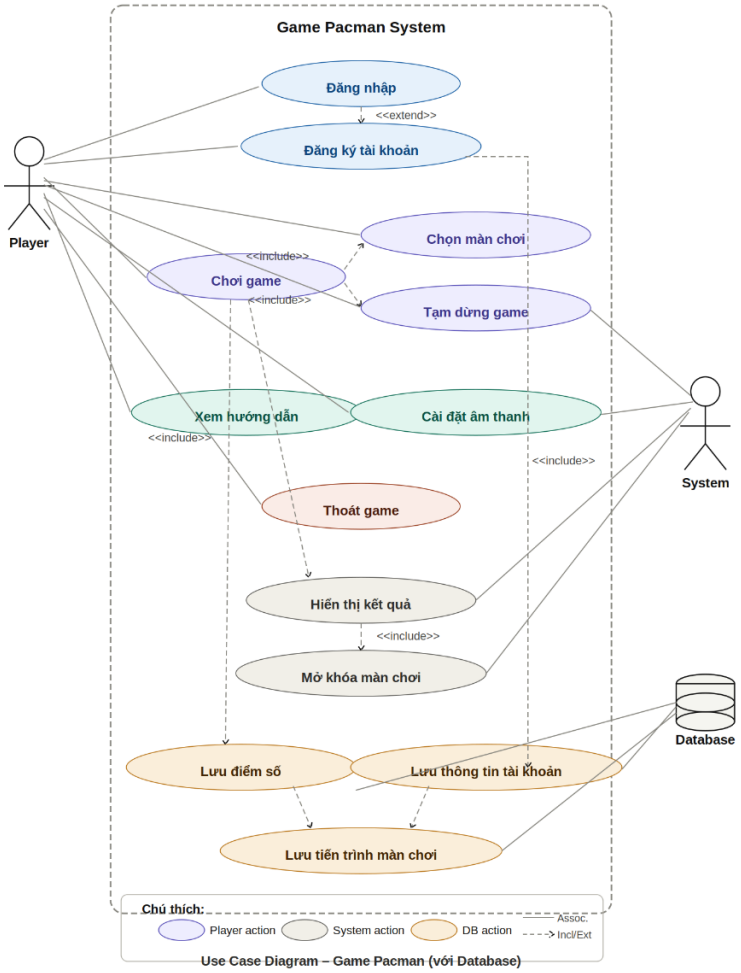
Hình 2. 1: Mô tả thiết lập của bản đồ, nhân vật, vật thể

2.2. Biểu đồ Use Case

Hệ thống được thiết kế xoay quanh hai Tác nhân chính: Người chơi và Cơ sở dữ liệu .

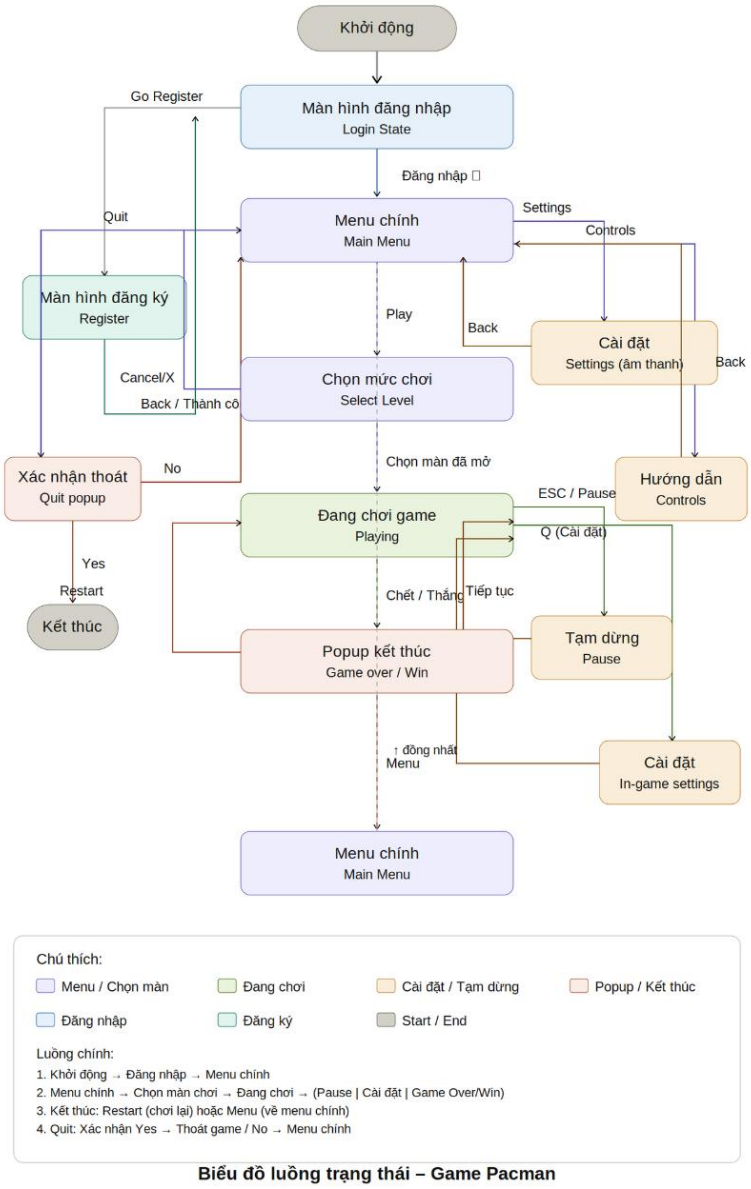
Với Người chơi: Các Use Case khởi đầu bao gồm Đăng ký (Register) và Đăng nhập (Login). Khi đã vào Sảnh chính (Main Menu), người chơi có thể Tùy chỉnh cài đặt (Settings - bật/tắt âm thanh). Xem hướng dẫn (Controls) hoặc Chọn màn chơi (Select Level).

Với Hệ thống ngầm: Khi người chơi hoàn thành một màn, Use Case "Cập nhật tiến trình" sẽ được tự động kích hoạt. Hệ thống sẽ giao tiếp với Database để so sánh điểm số hiện tại với kỷ lục cũ (High Score), lưu điểm mới và tự động tạo bản ghi mở khóa màn chơi tiếp theo (Level + 1).



Hình 2. 2: Biểu đồ use case của chương trình

2.3. Thiết kế luồng hoạt động



Hình 2. 3: Biểu đồ luồng hoạt động của chương trình

2.3.1. Luồng xác thực người dùng (Login / Register)

An toàn dữ liệu là một yếu tố được nhóm lưu tâm. Trong luồng Đăng ký, khi người dùng nhập tài khoản và mật khẩu, mật khẩu sẽ không được lưu dưới dạng chuỗi văn bản thuần. Thay vào đó, hệ thống sử dụng thuật toán băm SHA-256 để mã hóa mật khẩu trước khi lưu vào MySQL. Ở luồng Đăng nhập, hệ thống sẽ tiếp tục băm mật khẩu người dùng vừa nhập và đem so sánh với chuỗi mã hóa trong Database, đảm bảo tính bảo mật cơ bản cho trò chơi.

2.3.2. Luồng chọn màn chơi

Luồng này đóng vai trò kiểm soát nhịp độ trải nghiệm. Khi người chơi chọn một level (từ 1 đến 9) tại giao diện Start, một câu lệnh SQL sẽ được gọi để kiểm tra giới hạn level tối đa (maxUnlockedLevel) của user đó. Nếu người chơi chọn màn vượt quá giới hạn, một Popup cảnh báo "Level is Locked" sẽ hiện lên chặn lại. Nếu chọn màn hợp lệ,

một Popup "Level Info" sẽ xuất hiện, hiển thị điểm số họ từng đạt được ở màn này tạo động lực phá kỷ lục trước khi chính thức bấm "Play".

2.3.3. Luồng vòng lặp trò chơi (Game Loop)

Trái tim của hệ thống là vòng lặp Game Loop, được chạy trên một luồng (Thread) độc lập nhằm tránh gây đơ giao diện. Nhóm đã áp dụng thuật toán kiểm soát thời gian (Delta time) sử dụng System.nanoTime() để ép trò chơi chạy ở tốc độ khung hình cố định là 60 FPS (Frames Per Second). Vòng lặp này liên tục thực hiện 2 việc: Cập nhật tọa độ của Pacman/Ghost (Update) và vẽ lại toàn bộ màn hình. Trong quá trình này, luồng có thể bị tạm dừng nếu người dùng nhấn phím ESC (chuyển sang PAUSED), và sẽ hoàn toàn dừng lại khi chạm Ghost thì sẽ Game Over hoặc ăn hết Cherry thì sẽ Game Won.

2.4. Thiết kế trạng thái hệ thống

Để ứng dụng không bị rối loạn khi có quá nhiều giao diện khác nhau, nhóm áp dụng mô hình Máy trạng thái (State Machine) thông qua Enum GameState. Các trạng thái được quy định rõ ràng bao gồm: LOGIN, REGISTER, MAIN_MENU, START, PLAYING, PAUSED, SETTINGS, CONTROLS, QUIT_CONFIRM, LEVEL_LOCKED_POPUP, LEVEL_INFO_POPUP. Nhờ có biến trạng thái này, chương trình luôn biết chính xác người chơi đang ở đâu để tải đúng bộ nút bấm (MenuButton), phát đúng loại nhạc nền khi có thể chuyển qua lại giữa nhạc sảnh và nhạc chơi game, và render đúng màn hình.

2.5. Phân tích kiến trúc tổ chức

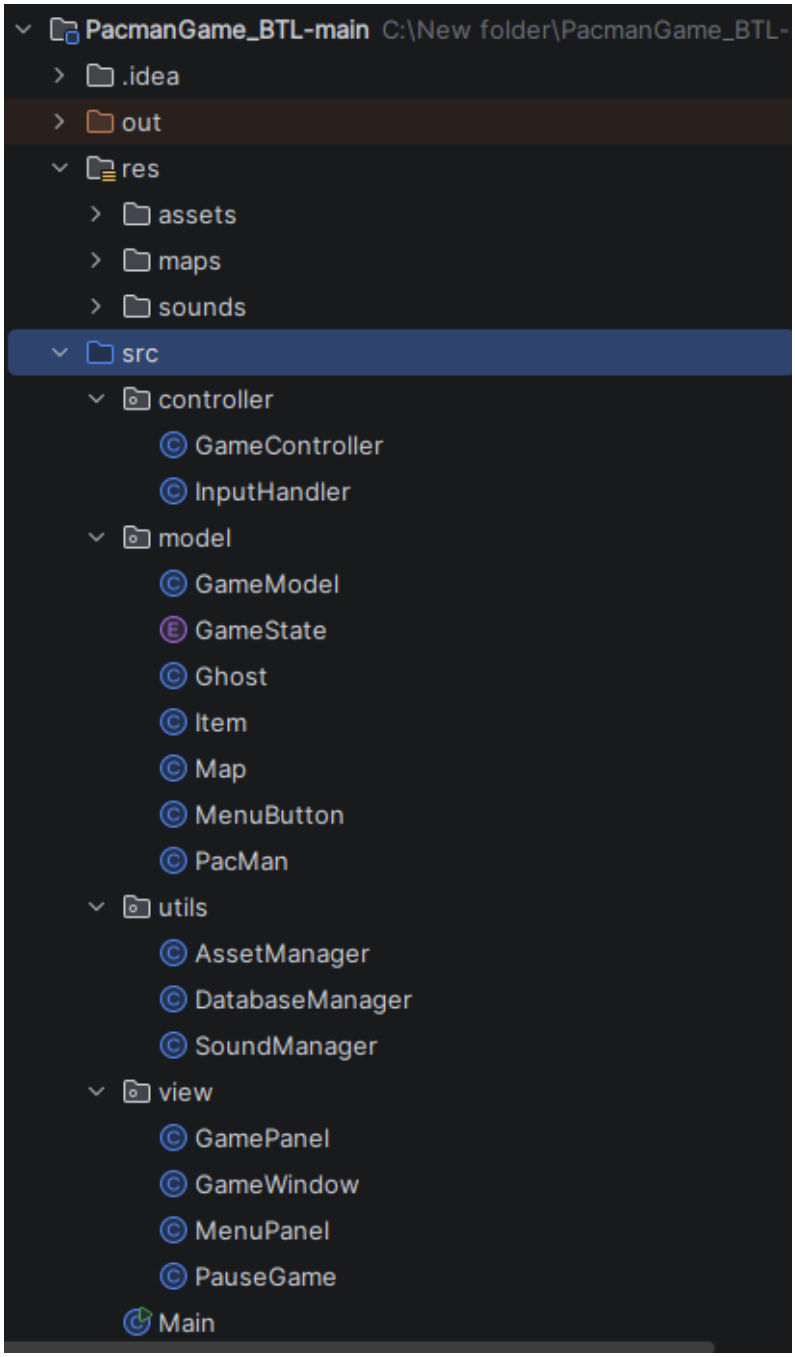
Dự án áp dụng triệt để mô hình kiến trúc phần mềm MVC (Model - View - Controller) kết hợp với các lớp Tiện ích (Utils) nhằm giúp code dễ bảo trì, dễ tìm lỗi và có khả năng mở rộng tốt. Cụ thể:

- Package Model: Nắm giữ toàn bộ dữ liệu. Bao gồm GameModel (lưu trữ điểm số, thông tin user hiện tại), Map (chứa ma trận lưới), và các thực thể như PacMan, Ghost, Item, MenuButton. Model hoàn toàn không biết gì về giao diện, nó chỉ quan tâm đến các con số tọa độ x, y và tốc độ (speed).

- Package View: Chịu trách nhiệm hiển thị. Lớp GameWindow đóng vai trò là khung viền cửa sổ, trong khi GamePanel là nơi trực tiếp dùng cọ vẽ (Graphics) để tô vẽ các ma trận chữ cái thành hình ảnh bức tường, nhân vật và giao diện UI tùy theo trạng thái GameState hiện tại.

- Package Controller: Bộ não điều phối. GameController chứa vòng lặp 60 FPS, nó tính toán đường đi, bắt sự kiện chạm viền lưới, xử lý logic ăn điểm và điều phối sự thay đổi của Model. Lớp InputHandler hỗ trợ Controller bằng cách lắng nghe các thao tác bấm phím (WASD) và click chuột.

- Package Utils: Nhóm đã áp dụng Design Pattern Singleton cho các class tại đây (AssetManager, SoundManager, DatabaseManager). Việc này đảm bảo hệ thống chỉ tạo ra đúng một bản sao duy nhất trong RAM để load hàng loạt hình ảnh, âm thanh và giữ kết nối Database. Nhờ vậy, game không phải load lại file ảnh mỗi khi vẽ một khung hình mới, giúp tối ưu hóa hiệu năng (Performance) một cách rõ rệt.



Hình 2. 4: Cây thư mục của chương trình

CHƯƠNG 3: CÀI ĐẶT CHI TIẾT

3.1. Lớp Model – Dữ liệu và trạng thái

3.1.1. Class Map

Lớp Map chịu trách nhiệm đọc và lưu trữ bản đồ dưới dạng ma trận kí tự hai chiều, cung cấp dữ liệu cho cả việc vẽ mê cung lẫn xác định vị trí ban đầu của các thực thể trong game.

- Bảng các thuộc tính trong class Map:

Map
Attributes
char[][] grid String currentLevel

Hình 3. 1: Bảng các thuộc tính trong class Map

grid: Ma trận char[ ][ ] lưu toàn bộ nội dung bản đồ. Đây là nguồn dữ liệu duy nhất cho việc vẽ bản đồ trong GamePanel và kiểm tra va chạm tường trong canMove().

currentLevel: Lưu tên file bản đồ hiện tại ("level1", "level2", ...) để GameController biết đang chơi màn nào khi cần restart hoặc chuyển màn tiếp theo.

- Bảng các phương thức trong class Map:

Map	
Methods	
Map(String mapFileName)	char[][] getGrid()
int getRows()	int getCols()
String getCurrentLevel()	void setCurrentLevel(String)

Hình 3. 2: Bảng các phương thức trong class Map

Map(): Gọi AssetManager.getInstance().getMap(mapFileName) để lấy bản đồ đã cache, sau đó clone từng dòng (grid[i] = cacheGrid[i].clone()) thay vì tham chiếu trực tiếp. Điều này đảm bảo mỗi màn chơi có bản sao độc lập – khi spawnEntities() xóa ký tự spawn thành ‘ ’, bản đồ gốc trong cache không bị ảnh hưởng, cho phép load lại màn chơi nhiều lần mà không cần đọc lại file.

getGrid(): Trả về tham chiếu trực tiếp đến grid[ ][ ]. GameController dùng để kiểm tra ký tự ‘X’ trong canMove(), spawnEntities() dùng để duyệt và thay ký tự spawn.

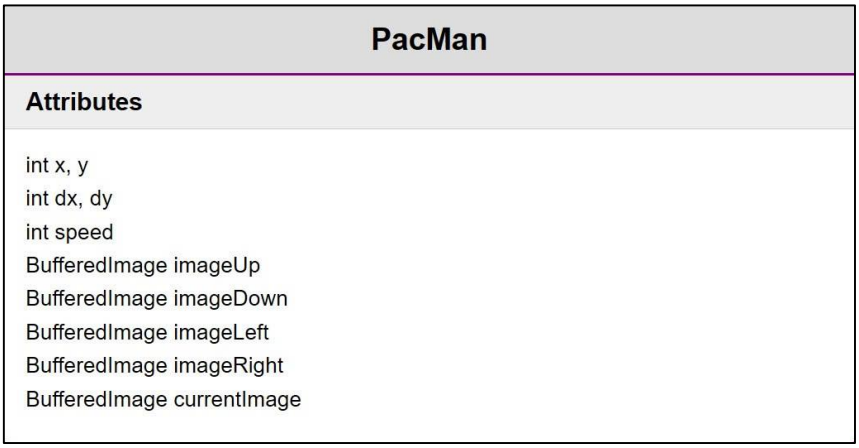
getRows(), getCols(): Trả về số hàng và số cột của bản đồ với kiểm tra null an toàn trước khi truy cập grid, tránh NullPointerException nếu file bản đồ load thất bại.

getCurrentLevel(), setCurrentLevel(): Getter/setter cho tên màn hiện tại. GameController đọc qua getCurrentLevel() khi cần xác định tên file để restart hoặc tính màn tiếp theo.

3.1.2. Class PacMan

Lớp PacMan lưu trữ toàn bộ dữ liệu của nhân vật chính trong game, bao gồm vị trí hiện tại, hướng và tốc độ di chuyển, cùng các hình ảnh hiển thị tương ứng với từng hướng. Tại lớp này, mọi logic điều khiển và xử lý va chạm đều do GameController đảm nhận, lớp PacMan chỉ cung cấp dữ liệu và cập nhật vị trí theo hướng hiện tại.

- Bảng các thuộc tính trong class PacMan:



Hình 3. 3: Bảng các thuộc tính trong class PacMan

x, y: Tọa độ pixel góc trên bên trái của sprite. Được tính từ vị trí ô lưới nhân với TILE_SIZE khi bắt đầu chơi, sau đó cập nhật liên tục mỗi frame thông qua update().

dx, dy: Vector hướng di chuyển, mỗi chiều nhận giá trị -1, 0 hoặc 1.

Bảng hướng di chuyển:

dx	dy	Hướng di chuyển
1	0	Sang phải
-1	0	Sang trái
0	-1	Lên trên
0	1	Xuống dưới
0	0	Đứng yên

speed = 2: PacMan di chuyển 2 pixel mỗi frame, nhanh hơn Ghost (speed = 1) để người chơi có thể thoát khỏi tình huống nguy hiểm.

imageUp/Down/Left/Right: Bốn ảnh tương ứng với bốn hướng di chuyển, load sẵn từ AssetManager trong constructor. Việc load trước toàn bộ ảnh tránh giật lag khi đổi hướng trong lúc chơi.

currentImage: Ảnh đang được hiển thị tại frame hiện tại. PacMan mặc định là imageRight khi bắt đầu. GamePanel chỉ cần gọi getCurrentImage() để lấy ảnh cần vẽ mà không cần biết PacMan đang đi hướng nào.

- Bảng các phương thức trong class PacMan:

PacMan	
Methods	
PacMan(int x, int y)	void update()
void setDirection(int dx, int dy)	BufferedImage getCurrentImage()

Hình 3. 4: Bảng các phương thức trong class PacMan

PacMan(int x, int y): Khởi tạo tọa độ, đặt speed = 2, dx = 0, dy = 0 (đứng yên), load 4 ảnh từ AssetManager và cài đặt currentImage = imageRight làm mặc định.

update(): Cộng (dx * speed) vào x và (dy * speed) vào y mỗi frame. Chỉ được GameController gọi sau khi canMove() xác nhận vị trí tiếp theo hợp lệ, đảm bảo PacMan không bao giờ đi xuyên tường.

setDirection(int dx, int dy): Cập nhật dx, dy và tự động chọn ảnh tương ứng gán vào currentImage. Nhờ vậy GamePanel không cần logic rẽ nhánh khi vẽ.

getCurrentImage(): Trả về currentImage để GamePanel vẽ lên màn hình mỗi frame.

3.1.3. Class Ghost

Lớp Ghost lưu trữ dữ liệu của ghost trong game, bao gồm vị trí, hướng di chuyển, tốc độ và hình ảnh hiển thị. Tương tự PacMan, lớp này thuần dữ liệu, logic AI điều khiển hướng di chuyển của Ghost được xử lý hoàn toàn bên trong GameController.

- Bảng các thuộc tính trong class Ghost:

Ghost
Attributes
int x, y int dx, dy int speed BufferedImage image

Hình 3. 5: Bảng các thuộc tính trong class Ghost

x, y: Tọa độ pixel góc trên bên trái của ảnh, cập nhật mỗi frame thông qua move().

dx, dy: Vector hướng di chuyển, tương tự PacMan. Hướng mặc định khi bắt đầu là dx=1, dy=0 (sang phải), GameController sẽ điều chỉnh ngay frame đầu nếu hướng này bị chặn.

speed = 1: Ghost di chuyển 1 pixel mỗi frame, chậm hơn PacMan (speed = 2) để đảm bảo cân bằng gameplay, người chơi luôn có khả năng thoát nếu phản ứng kịp.

image: Mỗi ghost có một ảnh riêng (redGhost, pinkGhost, orangeGhost, blueGhost) load qua AssetManager với tên do spawnEntities() truyền vào dựa theo ký tự bản đồ ('r', 'p', 'o', 'b').

- Bảng các phương thức trong class Ghost:

Ghost	
Methods	
Ghost(int x, int y, String imageName)	void move()
void setDirection(int dx, int dy)	

Hình 3. 6: Bảng các phương thức trong class Ghost

Ghost(int x, int y, String imageName): Khởi tạo tọa độ spawn, đặt speed = 1, dx = 1, dy = 0, load ảnh từ AssetManager theo imageName.

move(): Cộng (dx * speed) vào x và (dy * speed) vào y. Chỉ được GameController gọi trong moveGhost() sau khi canMove() xác nhận hợp lệ.

setDirection(int dx, int dy): Cập nhật hướng di chuyển. Được GameController gọi trong changeGhostDirection() sau khi thuật toán AI chọn ra hướng mới.

3.1.4. Class Item

Lớp Item đại diện cho các vật phẩm xuất hiện trên bản đồ mà PacMan có thể thu thập để ghi điểm. Trong game hiện tại, vật phẩm là Cherry với giá trị 10 điểm mỗi lần ăn.

- Bảng các thuộc tính trong class Item:

Item
Attributes
int x, y String type int scoreValue BufferedImage image

Hình 3. 7: Bảng các thuộc tính trong class Item

x, y: Tọa độ pixel cố định từ lúc bắt đầu, không thay vì item không di chuyển. Item không có setter cho x, y – khi bị ăn, item được xóa thẳng khỏi List<Item> trong GameModel.

type: Loại vật phẩm, hiện tại chỉ có "cherry". Dự phòng để mở rộng thêm loại vật phẩm khác trong tương lai mà không cần thay đổi cấu trúc lớp.

scoreValue = 10: Số điểm cộng vào score khi PacMan ăn item này.

image: Ảnh của item, load từ AssetManager theo imageName khi khởi tạo.

- Bảng các phương thức trong class Item:

Item	
Methods	
Item(int x, int y, String type, int scoreValue, String imageName)	Rectangle getBounds()

Hình 3. 8: Bảng các phương thức trong class Item

Item(int x, int y, String type, int scoreValue, String imageName): Khởi tạo toàn bộ thuộc tính, load ảnh từ AssetManager. Được spawnEntities() gọi với tham số cố định: type = "cherry", scoreValue = 10, imageName = "cherry".

getBounds(): Trả về new Rectangle(x, y, TILE_SIZE, TILE_SIZE) – vùng va chạm đúng bằng một ô lưới 32×32 pixel. GameController dùng intersects() giữa bounds thu nhỏ của PacMan và bounds của item để phát hiện va chạm mỗi frame.

3.1.5. Class MenuButton

Lớp MenuButton đại diện cho các nút bấm xuất hiện trên toàn giao diện game, từ màn hình đăng nhập, menu chính cho đến màn hình chọn level. Ngoài việc lưu trữ thông tin hiển thị, lớp còn tích hợp cơ chế animation phóng to khi người dùng di chuột vào.

- Bảng các thuôc tính trong class MenuButton:

MenuButton
Attributes
int x, y, width, height BufferedImage image String actionCommand boolean hovered float currentScale static final float HOVER_SCALE = 1.12f static final float SCALE_SPEED = 0.08f

Hình 3. 9: Bảng các thuộc tính trong class MenuButton

x, y, width, height: Tọa độ và kích thước gốc của button tính bằng pixel. Được dùng làm vùng kiểm tra click và hover, không thay đổi dù button đang scale lên hay xuống.

image: Ảnh của button load từ AssetManager. Một số button đặc biệt như TOGGLE_PASSWORD có thể thay ảnh runtime qua setImage() khi trạng thái thay đổi.

actionCommand: Chuỗi định danh hành động ("PLAY", "RESUME", "RESTART", ...). Khi người dùng click, GameController lấy actionCommand và xử lý trong handleButtonClick().

hovered: Thuộc tính trạng thái hover, được GameController cập nhật mỗi khi chuột di chuyển qua handleMouseMove(). updateHoverAnimation() đọc thuộc tính này để quyết định tăng hay giảm currentScale.

currentScale: Tỉ lệ phóng to hiện tại của button, dao động giữa 1.0f và HOVER_SCALE. Được cập nhật dần dần mỗi frame tạo hiệu ứng mượt mà.

HOVER_SCALE = 1.12f: Button phóng to 12% khi hover. Đủ lớn để người dùng nhận ra phản hồi nhưng không gây rối giao diện.

SCALE_SPEED = 0.08f: Tốc độ thay đổi scale mỗi frame. Với 60 FPS, button mất khoảng 2 frame để đạt kích thước tối đa.

- Bảng các phương thức class MenuButton:

MenuButton	
Methods	
MenuButton(int x, int y, int width, int height, BufferedImage image, String actionCommand)	
void updateHoverAnimation()	void draw(Graphics g)
boolean isClicked(int mouseX, int mouseY)	boolean contains(int mouseX, int mouseY)
void setHovered(boolean)	boolean isHovered()
String getActionCommand()	void setImage(BufferedImage)

Hình 3. 10: Bảng các phương thức trong class MenuButton

MenuButton(): Khởi tạo toàn bộ thuộc tính, currentScale = 1.0f và hovered = false mặc định.

updateHoverAnimation(): Tính targetScale dựa theo hovered, nếu hover thì target là HOVER_SCALE, ngược lại là 1.0f. Tăng hoặc giảm currentScale về target với bước nhảy SCALE_SPEED mỗi frame bằng Math.min/max để không vượt quá giới hạn.

draw(Graphics g): Tính (drawWidth = width * currentScale), (drawHeight = height * currentScale), sau đó tính lại drawX = x - (drawWidth - width)/2 và drawY = y - (drawHeight - height)/2 để giữ nguyên tâm button khi phóng to. Bật RenderingHints.VALUE_INTERPOLATION_BILINEAR để ảnh mượt hơn khi scale.

isClicked(int mouseX, int mouseY): Dùng Rectangle(x, y, width, height).contains(mouseX, mouseY) để kiểm tra trên vùng gốc chưa scale để vùng click không bị lệch khi button đang phóng to.

contains(int mouseX, int mouseY): Tương tự isClicked(), kiểm tra vùng gốc. Được GameController dùng trong handleMouseMove() để cập nhật trạng thái hovered.

setImage(BufferedImage): Cho phép thay ảnh button runtime. Dùng cho button TOGGLE_PASSWORD, khi người dùng nhấn, ảnh chuyển giữa hidePassword và viewPassword để phản ánh trạng thái hiện tại.

3.1.6. Class GameModel

GameModel là tầng Model trong MVC, không chứa logic xử lý hay hiển thị, chỉ lưu trữ toàn bộ dữ liệu để GameController và GamePanel cùng tham chiếu.

- Bảng các thuộc tính trong class GameModel:

GameModel	
Attributes	
static final int TILE_SIZE	Map map
List<Ghost> ghosts	List<Item> items
PacMan pacman	int score
GameState currentState	boolean isGameOver
boolean isGameWon	String currentUser
int maxUnlockedLevel	int selectedLevelId
int selectedLevelHighScore	String loginUsername
String loginPassword	String registerUsername
String registerPassword	String loginMessage
int activeField	boolean showPassword
List<MenuButton> mainMenuButtons	List<MenuButton> pauseButtons
List<MenuButton> quitConfirmButtons	List<MenuButton> startButtons
List<MenuButton> settingsButtons	List<MenuButton> gamePlayButtons
List<MenuButton> gameOverButtons	List<MenuButton> gameWonButtons
List<MenuButton> loginButtons	List<MenuButton> registerButtons
List<MenuButton> levelLockedButtons	List<MenuButton> levelInfoButtons
MenuButton controlsButtons	

Hình 3. 11: Bảng các thuộc tính trong class GameModel

TILE_SIZE = 32: Mỗi ô trên bản đồ có kích thước 32×32 pixel.

map, pacman, ghosts, items: Lưu toàn bộ đối tượng đang tồn tại trong màn chơi hiện tại. Được khởi tạo và cập nhật mỗi khi bắt đầu màn mới thông qua setMap() và spawnEntities().

score, isGameOver, isGameWon: score được cộng dồn mỗi khi PacMan ăn item. isGameOver và isGameWon là hai biến trạng thái do GameController cập nhật sau mỗi frame kiểm tra, GamePanel đọc để quyết định vẽ overlay nào lên màn hình.

currentUser, maxUnlockedLevel, selectedLevelId, selectedLevelHighScore: Dùng để lưu tiến trình người dùng. maxUnlockedLevel được load từ database sau khi đăng nhập và cập nhật mỗi khi hoàn thành màn mới. selectedLevelId và selectedLevelHighScore là dữ liệu tạm thời phục vụ popup LEVEL_INFO_POPUP trước khi vào chơi.

loginUsername, loginPassword, activeField, showPassword, loginMessage: Lưu trạng thái tức thời của form đăng nhập. activeField (0 = username, 1 = password) xác định ô đang được focus, chuyển đổi bằng TAB hoặc click chuột. showPassword khi false thì GamePanel vẽ ‘ * ’ thay ký tự thật. loginMessage hiển thị thông báo lỗi hoặc thành công lên màn hình.

Các List<MenuButton>: Mỗi màn hình có một list button riêng khởi tạo sẵn trong initMenus(). GamePanel dùng để vẽ, GameController dùng để kiểm tra click và hover – cả hai cùng trỏ vào một list duy nhất trong Model.

- Bảng các phương thức trong class GameModel:

GameModel	
Methods	
GameModel()	void loadBackgrounds()
void initMenus()	void setMap(Map map)
void spawnEntities()	void resetGame(String levelName)
void addScore(int points)	GameState getCurrentState()
void setCurrentState(GameState)	boolean isGameOver()
void setGameOver(boolean)	boolean isGameWon()
void setGameWon(boolean)	

Hình 3. 12: Bảng các phương thức trong class GameModel

GameModel(): Constructor khởi tạo các ArrayList rỗng cho ghosts và items, sau đó gọi loadBackgrounds() để cache toàn bộ ảnh nền và initMenus() để tạo sẵn tất cả button cho mọi màn hình.

loadBackgrounds(): Gọi AssetManager.getInstance().getImage() cho từng ảnh nền. Mục đích không phải để lưu ảnh vào biến mà để cache trước – lần đầu gọi getImage() sẽ đọc file từ disk và lưu vào cache, các lần sau trả về ngay từ bộ nhớ, tránh giật lag khi chuyển màn hình.

setMap(Map map): Gán bản đồ mới vào model rồi gọi ngay spawnEntities() để khởi tạo lại toàn bộ thực thể trên bản đồ đó.

spawnEntities(): Xóa sạch ghosts và items, reset score = 0, sau đó duyệt toàn bộ char[ ][ ] để tạo đối tượng tương ứng.

resetGame(String levelName): Tạo Map mới từ levelName, gọi setMap() để khởi tạo lại toàn bộ thực thể, reset isGameOver = false và isGameWon = false.

addScore(int points): Cộng points vào score hiện tại. Được GameController gọi mỗi khi PacMan ăn item.

getCurrentState(), setCurrentState(): Getter/setter cho currentState. setCurrentState() là điểm duy nhất thay đổi trạng thái game – GameController gọi khi cần chuyển màn hình, GamePanel đọc getCurrentState() mỗi frame để quyết định vẽ giao diện nào.

isGameOver(), setGameOver(), isGameWon(), setGameWon(): Getter/setter cho hai biến trạng thái kết thúc game. GameController cập nhật sau khi kiểm tra va chạm hoặc điều kiện thắng, GamePanel đọc để hiển thị overlay Game Over hoặc Game Won đúng lúc.

3.2. Lớp Controller – Điều phối logic

Trong kiến trúc MVC của game, lớp Controller đóng vai trò trung tâm điều phối toàn bộ logic của game. Lớp Controller nhận sự kiện từ InputHandler, cập nhật dữ liệu trong GameModel và kích hoạt GamePanel vẽ lại giao diện mỗi frame. Mọi quyết định về va chạm, di chuyển, AI Ghost hay chuyển cảnh đều được xử lý tại đây.

3.2.1. Vòng lặp game

- Bảng thuộc tính trong class GameController:

GameController	
Attributes	
GameModel model	GamePanel viewPanel
GameWindow window	SoundManager sound
String crmap	GameState previousState
GameState stateBeforeSettings	int bufferedDx, bufferedDy
Thread gameThread	boolean isRunning
final int FPS = 60	Random random

Hình 3. 13: Bảng các thuộc tính trong class GameController

model: Tham chiếu đến GameModel, nguồn dữ liệu duy nhất của hệ thống. Mọi đọc/ghi dữ liệu đều thông qua model, không có biến trạng thái nào lưu riêng trong GameController.

viewPanel: Tham chiếu đến GamePanel để gọi repaint() mỗi frame. GameController không trực tiếp vẽ bất cứ thứ gì, chỉ báo cho GamePanel biết cần vẽ lại.

sound: Instance của SoundManager lấy qua Singleton. Được dùng để phát/dừng âm thanh khi xảy ra sự kiện như ăn cherry, va chạm ghost, thắng/thua màn.

crmap: Lưu tên bản đồ hiện tại ("level1", "level2", ...). Khi người chơi nhấn Restart, startLevel(crmap) được gọi để load lại đúng màn đang chơi.

previousState: Lưu trạng thái của frame trước, dùng trong handleStateSound() để phát hiện khi nào trạng thái vừa thay đổi và xử lý chuyển nhạc tương ứng.

stateBeforeSettings: Lưu trạng thái trước khi vào Settings vì Settings có thể mở từ cả PLAYING lẫn MAIN_MENU, khi thoát Settings cần biết quay về đâu.

bufferedDx, bufferedDy: Lưu hướng di chuyển người chơi vừa nhấn phím, không áp dụng ngay mà chờ đến update() tiếp theo để xử lý theo cơ chế queued turn.

FPS = 60: Số frame mỗi giây. Từ đây tính ra drawInterval = 1 000 000 000 / 60 nanosecond, khoảng thời gian tối thiểu giữa hai frame liên tiếp.

random: Đối tượng Random khởi tạo một lần, dùng chung cho tất cả ghost khi chọn hướng ngẫu nhiên trong changeGhostDirection().

- Bảng phương thức trong class GameController:

GameController	
Methods	
GameController()	void startGameLoop()
void run()	void update()

Hình 3. 14: Bảng các phương thức trong class GameController

GameController(): Khởi tạo GameModel, lấy instance SoundManager và load toàn bộ âm thanh, tạo GamePanel, gắn InputHandler, mở GameWindow rồi gọi startGameLoop().

startGameLoop(): Tạo Thread mới với this là Runnable, gán vào gameThread, gọi start() để bắt đầu vòng lặp game trên thread riêng biệt với UI thread của Swing.

run(): Vòng lặp chính chạy liên tục khi isRunning = true. Mỗi vòng tính thời gian đã trôi qua kể từ frame trước, tích lũy vào delta, khi delta >= 1 thì gọi update(), updateHoverAnimations(), viewPanel.repaint() rồi trừ delta đi 1. Cách này đảm bảo logic luôn chạy đúng 60 lần/giây bất kể tốc độ máy.

update(): Gọi handleStateSound() mỗi frame, sau đó chỉ xử lý logic gameplay khi currentState == PLAYING và game chưa kết thúc. Lần lượt xử lý queued turn của PacMan, di chuyển PacMan, di chuyển toàn bộ ghost, kiểm tra va chạm và điều kiện thắng/thua.

3.2.2. Thuật toán kiểm tra va chạm

- Bảng thuộc tính trong class GameController:

GameController	
Attributes	
final int tileSize = 32	final int offset = 2
final int maxCols = 19	final int maxRows = 21

Hình 3. 15: Bảng các thuộc tính liên quan đến canMove

tileSize = 32: Kích thước một ô lưới tính bằng pixel, dùng để chuyển đổi tọa độ pixel sang tọa độ ô lưới bằng phép chia nguyên.

offset = 2: Thu nhỏ vùng kiểm tra 2px mỗi bên giúp nhân vật đi sát tường mà không bị kẹt do sai số pixel khi di chuyển với speed = 2.

maxCols = 19, maxRows = 21: Giới hạn của bản đồ. Kiểm tra trước khi tra grid[][] để tránh ArrayIndexOutOfBoundsException khi nhân vật chạm biên.

- Bảng phương thức trong class GameController:

GameController
Methods
boolean canMove(int nextX, int nextY)

Hình 3. 16: Bảng các phương thức liên quan đến canMove

canMove(int nextX, int nextY): Tính 4 góc của ảnh sau khi thu nhỏ vào offset pixel:

leftCol = (nextX + offset) / tileSize

rightCol = (nextX + tileSize - offset - 1) / tileSize

topRow = (nextY + offset) / tileSize

bottomRow = (nextY + tileSize - offset - 1) / tileSize

Trước tiên kiểm tra giới hạn, nếu bất kỳ cột nào < 0 hoặc ≥ 19, hàng nào < 0 hoặc ≥ 21 thì trả về false ngay. Sau đó tra bảng grid[][] tại cả 4 góc, nếu bất kỳ góc nào rơi vào ô 'X' thì trả về false. Chỉ khi cả 4 góc đều hợp lệ mới trả về true.

3.2.3. Di chuyển Pacman

- Bảng thuộc tính của class GameController:

GameController
Attributes
int bufferedDx int bufferedDy

Hình 3. 17: Bảng các thuộc tính liên quan đến queued turn

Khi người chơi nhấn phím, hướng mong muốn được lưu vào bufferedDx, bufferedDy thay vì áp dụng ngay, tránh tình trạng PacMan bị kẹt vào tường do rẽ sai thời điểm.

bufferedDx, bufferedDy: Hướng di chuyển người chơi vừa nhấn phím, giữ nguyên giá trị qua nhiều frame cho đến khi được áp dụng thành công hoặc bị ghi đè bởi phím mới.

- Bảng phương thức của class GameController:

GameController
Methods
void handleKeyPress(int key) void update()

Hình 3. 18: Bảng các phương thức liên quan đến queued turn

handleKeyPress(int key): Khi người chơi nhấn phím mũi tên hoặc 'WASD' trong trạng thái PLAYING, gán hướng tương ứng vào bufferedDx, bufferedDy.

update(): Mỗi frame xử lý bufferedDx, bufferedDy theo hai trường hợp:

* Quay đầu 180°: Nếu hướng buffer ngược hoàn toàn với hướng hiện tại, áp dụng ngay lập tức tại bất kỳ vị trí nào miễn canMove() hợp lệ, phản ứng tức thì khi người chơi muốn chạy ngược lại.

* Rẽ 90°: Chỉ áp dụng khi PacMan đang đứng đúng tâm ô và đường rẽ thông thoáng. Nếu chưa đến tâm ô, bufferedDx, bufferedDy giữ nguyên để frame tiếp theo thử lại, cho phép người chơi nhấn phím trước khi đến góc cua mà vẫn rẽ được đúng lúc.

* Sau khi áp dụng thành công, bufferedDx = 0 và bufferedDy = 0 để tránh hướng cũ bị áp dụng lại ở frame sau.

3.2.4. Thuật toán AI Ghost

AI Ghost sử dụng thuật toán di chuyển ngẫu nhiên có ràng buộc, không quay đầu 180° trừ khi bắt buộc, tạo chuyển động tự nhiên và khó đoán.

- Bảng thuộc tính:

GameController
Attributes
Random random

Hình 3. 19: Bảng các thuộc tính liên quan đến AI Ghost

random: Đối tượng Random dùng chung cho tất cả ghost. Mỗi lần changeGhostDirection() được gọi, random.nextInt(validMoves.size()) chọn ngẫu nhiên một hướng hợp lệ từ danh sách.

- Bảng phương thức:

GameController
Methods
void moveGhost(Ghost ghost) void changeGhostDirection(Ghost ghost)

Hình 3. 20: Bảng các phương thức liên quan đến AI Ghost

moveGhost(Ghost ghost): Mỗi frame thử di chuyển ghost theo hướng hiện tại bằng canMove(). Nếu đường bị chặn, đưa ghost về tâm ô rồi gọi changeGhostDirection. Khi đứng đúng tâm ô, ghost cũng có thể đổi hướng ngẫu nhiên ngay cả khi đường thẳng vẫn đi được, tạo chuyển động không đoán trước.

changeGhostDirection(Ghost ghost): Liệt kê 4 hướng {0,-1}, {0,1}, {-1,0}, {1,0}, loại bỏ hướng ngược 180, lọc các hướng canMove hợp lệ từ tâm ô hiện tại (baseX = (x/tileSize)*tileSize), rồi chọn ngẫu nhiên bằng random.nextInt(). Nếu validMoves rỗng, ghost mới được phép quay đầu 180°, đây là trường hợp duy nhất ghost quay ngược chiều.

3.2.5. Xử lý va chạm PacMan – Ghost và PacMan – Item

- Bảng phương thức:

GameController
Methods
void checkEatCherry() void checkGhostCollision()

Hình 3. 21: Bảng các phương thức xử lý va chạm

checkEatCherry(): Tạo Rectangle thu nhỏ của PacMan với padding 8px mỗi bên. Duyệt List<Item> bằng Iterator, nếu intersects(item.getBounds()) thì cộng scoreValue vào điểm, xóa item khỏi danh sách và phát eat.wav.

checkGhostCollision(): Tạo Rectangle thu nhỏ 4px của cả PacMan lẫn từng ghost. Nếu intersects() thì cập nhật isGameOver = true, dừng nhạc nền, phát game_loss.wav và break ngay để tránh xử lý nhiều va chạm cùng lúc.

Item dùng padding 8px vì cherry nhỏ, cần PacMan đến gần mới tính là ăn. Ghost dùng padding 4px vì va chạm với ghost cần phản ánh đúng thực tế hình ảnh hơn.

3.2.6. Điều kiện thắng/thua và chuyển màn

- Bảng phương thức:

GameController
Methods
void checkWinCondition() void startLevel(String levelName) void goToNextLevel() String getMapName(int lvl)

Hình 3. 22: Bảng các phương thức xử lý thắng/thua và chuyển màn

checkWinCondition(): Nếu model.getItems().isEmpty() (hết toàn bộ cherry) và chưa thắng thì cập nhật isGameWon = true, dừng nhạc nền, phát game_win.wav. Nếu người dùng đang đăng nhập thì gọi DatabaseManager.updateScoreAndUnlockNext() để lưu điểm cao và mở khóa màn tiếp theo, đồng thời cập nhật maxUnlockedLevel ngay trên model để giao diện phản ánh kịp thời.

startLevel(String levelName): Gán crmap = levelName, tạo Map mới và gọi model.setMap() để bắt đầu lại toàn bộ thực thể, cập nhật isGameOver = false và isGameWon = false, dừng tất cả âm thanh đang loop, chuyển sang PLAYING và bắt đầu phát background.wav.

goToNextLevel(): Dùng switch(crmap) để xác định tên màn tiếp theo từ màn hiện tại rồi gọi startLevel(). Hiện tại game có 3 bản đồ xoay vòng: leveltest → level2 → level3 → level1.

getMapName(int lvl): Map số màn (1–9) sang tên file bằng lvl % 3 — mod 1 → "leveltest", mod 2 → "level2", mod 0 → "level3".

3.2.7. Xử lý chuyển cảnh theo GameState

- Bảng thuộc tính:

GameController
Attributes
GameState stateBeforeSettings GameState previousState

Hình 3. 23: Bảng các thuộc tính liên quan đến chuyển cảnh

stateBeforeSettings: Lưu trạng thái trước khi vào Settings. Khi nhấn BACK_TO_MENU từ Settings, hệ thống kiểm tra stateBeforeSettings, nếu là PLAYING thì quay về PLAYING, ngược lại về MAIN_MENU.

previousState: Lưu trạng thái của frame trước để handleStateSound() phát hiện khi nào trạng thái vừa thay đổi và xử lý chuyển nhạc.

- Bảng phương thức:

GameController	
Methods	
void handleButtonClick(String command)	String getMusicForState(GameState state)
void handleLevelSelection(int requestedLvl)	void handleMousePress(int mx, int my)
void handleStateSound()	List<MenuButton> getButtonsForState(GameState state)

Hình 3. 24: Bảng các phương thức liên quan đến chuyển cảnh

handleButtonClick(String command): Nhận actionCommand và xử lý qua switch-case. Mỗi case thực hiện chuyển GameState, reset dữ liệu hoặc gọi các phương thức liên quan.

handleMousePress(int mx, int my): Xác định danh sách button của trạng thái hiện tại, duyệt từng button gọi isClicked(mx, my), nếu đúng thì chuyển actionCommand sang handleButtonClick(). Trạng thái PLAYING cần kiểm tra thêm isGameOver và isGameWon để xác định đúng bộ button cần kiểm tra.

getButtonsForState(GameState state): Trả về List<MenuButton> tương ứng với trạng thái hiện tại qua switch-case. Được dùng chung cho cả handleMousePress() và handleMouseMove() để tránh lặp code.

handleLevelSelection(int requestedLvl): Nếu màn bị khóa thì chuyển sang LEVEL_LOCKED_POPUP. Ngược lại truy vấn điểm cao từ database, nếu đã từng chơi (highScore >= 0) thì hiện LEVEL_INFO_POPUP với điểm cao, nếu chưa thì gọi startLevel() luôn.

handleStateSound(): Gọi mỗi frame, so sánh previousState với currentState. Nếu khác nhau thì gọi getMusicForState() cho cả hai state, nếu cùng nhóm nhạc thì giữ nguyên không restart, nếu khác nhóm thì dừng nhạc cũ và phát nhạc mới.

getMusicForState(GameState state): Nhóm các state theo nhạc nền: MAIN_MENU/START/CONTROLS/QUIT_CONFIRM/LEVEL_LOCKED_POPUP/ LEVEL_INFO_POPUP trả về "menu.wav", PLAYING trả về "background.wav", các state còn lại (PAUSED, SETTINGS, LOGIN, REGISTER) trả về null.

3.2.8. InputHandler – Bắt sự kiện bàn phím, chuột, hover

InputHandler chịu trách nhiệm bắt toàn bộ sự kiện từ người dùng và chuyển tiếp sang GameController xử lý. InputHandler cung cấp ba listener tương ứng với ba loại sự kiện.

- Bảng thuộc tính trong class InputHandler:

InputHandler
Attributes
GameController controller

Hình 3. 25: Bảng các thuộc tính trong class InputHandler

controller: Tham chiếu đến GameController, dùng để chuyển tiếp toàn bộ sự kiện sang các phương thức xử lý tương ứng.

- Bảng phương thức trong class InputHandler:

InputHandler
Methods
InputHandler(GameController controller)
KeyAdapter getKeyListener()
MouseAdapter getMouseListener()
MouseMotionAdapter getMouseMotionListener()

Hình 3. 26: Bảng các phương thức trong class InputHandler

InputHandler(GameController controller): Nhận tham chiếu GameController và lưu vào controller. Không khởi tạo listener tại đây mà tạo theo yêu cầu qua các phương thức get*Listener().

getKeyListener(): Tạo KeyAdapter với hai phương thức: keyPressed dùng e.getKeyCode() để nhận phím đặc biệt chuyển sang controller.handleKeyPress(); keyTyped dùng e.getKeyChar() để nhận ký tự Unicode thực tế người dùng gõ chuyển sang controller.handleKeyTyped().

getMouseListener(): Tạo MouseAdapter xử lý mousePressed thay vì mouseClicked, mouseClicked yêu cầu nhấn và thả tại cùng vị trí nên dễ bị bỏ sót khi người dùng nhấn nhanh hoặc di chuột nhẹ khi nhấn. Tọa độ e.getX(), e.getY() được chuyển sang controller.handleMousePress().

getMouseMotionListener(): Tạo MouseMotionAdapter xử lý mouseMoved, mỗi khi chuột di chuyển, tọa độ được chuyển sang controller.handleMouseMove() để cập nhật trạng thái hovered của tất cả button trong frame hiện tại, phục vụ cho updateHoverAnimation().

3.3. Lớp View – Hiển thị đồ họa

Lớp View chịu trách nhiệm hiển thị toàn bộ giao diện game lên màn hình. Lớp View không chứa logic nào, chỉ đọc dữ liệu từ GameModel và vẽ lại mỗi frame.

3.3.1. GamePanel – Render toàn bộ giao diện theo GameState hiện tại

- Bảng thuộc tính trong class GamePanel:

GamePanel	
Attributes	
GameModel gameModel	static final int PANEL_WIDTH = 608
Font titleFont	static final int PANEL_HEIGHT = 672

Hình 3. 27: Bảng các thuộc tính trong class GamePanel

PANEL_WIDTH = 608, PANEL_HEIGHT = 672: Kích thước cố định của cửa sổ game, tương ứng với bản đồ 19×21 ô nhân với TILE_SIZE = 32 pixel. Được khai báo static final để GameWindow dùng khi tạo cửa sổ.

gameModel: Tham chiếu đến GameModel, nguồn dữ liệu duy nhất để GamePanel đọc và vẽ. GamePanel không được phép thay đổi bất kỳ dữ liệu nào trong GameModel.

titleFont: Font chữ Arial Bold 40pt dùng chung cho các tiêu đề màn hình, khai báo một lần để tránh tạo lại mỗi frame.

- Bảng phương thức trong class GamePanel:

GamePanel
Methods
GamePanel(GameModel gameModel)
void paintComponent(Graphics g)
void drawBackground(Graphics g, String imageName)
void drawOverlay(Graphics g, int alpha)

Hình 3. 28: Bảng các phương thức trong class GamePanel

GamePanel(GameModel gameModel): Nhận tham chiếu GameModel, thiết lập kích thước panel bằng setPreferredSize() và màu nền đen làm mặc định.

paintComponent(Graphics g): Phương thức render chính, được GameController kích hoạt gián tiếp qua viewPanel.repaint() mỗi frame. Đọc currentState từ GameModel và dùng switch-case để gọi phương thức vẽ tương ứng, mỗi trạng thái có một bộ phương thức vẽ riêng.

drawBackground(Graphics g, String imageName): Lấy ảnh nền từ AssetManager theo tên và vẽ full màn hình 608×672. Nếu ảnh không tồn tại thì vẽ nền đen thay thế.

DrawOverlay(Graphics g, int alpha): Vẽ một lớp màu đen bán trong suốt lên toàn màn hình. Dùng để làm tối nền khi hiện popup như LEVEL_LOCKED_POPUP, LEVEL_INFO_POPUP, Game Over, Game Won.

3.3.2. Vẽ bản đồ tile-by-tile từ ma trận ký tự

Bản đồ được vẽ bằng cách duyệt toàn bộ ma trận char[][] và vẽ từng ô một theo đúng tọa độ pixel.

- Bảng phương thức:

GamePanel
Methods
GamePanel(GameModel gameModel) void paintComponent(Graphics g) void drawBackground(Graphics g, String imageName) void drawOverlay(Graphics g, int alpha)

Hình 3. 29: Bảng các phương thức vẽ bản đồ và nhân vật

drawMap(Graphics g): Duyệt từng ô grid[row][col], tính tọa độ pixel x = col * TILE_SIZE, y = row * TILE_SIZE, sau đó vẽ ảnh tương ứng: 'X' vẽ ảnh tường, '.' ảnh cherry, ' ' ô trống không vẽ gì. Ảnh mỗi ô được vẽ đúng kích thước 32×32 pixel. Sau khi spawnEntities() xóa ký tự nhân vật thành ' ', bản đồ chỉ còn tường và cherry, PacMan, Ghost, Item được vẽ riêng trong drawEntities().

drawEntities(Graphics g): Vẽ PacMan bằng pacman.getCurrentImage() tại tọa độ (pacman.getX(), pacman.getY()). Sau đó duyệt List<Item> vẽ từng cherry còn lại trên bản đồ, rồi duyệt List<Ghost> vẽ từng ghost. Cả 3 loại thực thể đều được vẽ với kích thước TILE_SIZE × TILE_SIZE. Kiểm tra null trước khi vẽ để tránh NullPointerException nếu ảnh chưa load xong.

3.3.3. Vẽ các màn hình

- Bảng các phương thức:

GamePanel	
Methods	
void drawLoginScreen(Graphics g) void drawRegisterScreen(Graphics g) void drawMainMenu(Graphics g) void drawStartMenu(Graphics g) void drawLevelLockedPopup(Graphics g) void drawLevelInfoPopup(Graphics g)	void drawPauseMenu(Graphics g) void drawQuitConfirm(Graphics g) void drawGame(Graphics g) void drawGameOverScreen(Graphics g) void drawGameWonScreen(Graphics g) void drawSoundStatus(Graphics g, String soundStatus)

Hình 3. 30: Bảng các phương thức vẽ các màn hình

drawLoginScreen(): Vẽ hai ô input username và password bằng drawRect(). Ô đang được focus (activeField) có viền vàng, ô còn lại viền trắng. Nếu showPassword = false thì thay từng ký tự mật khẩu bằng '*' trước khi vẽ. Thông báo lỗi loginMessage được vẽ màu đỏ bên dưới. Sau đó duyệt loginButtons gọi btn.draw(g) để vẽ từng nút.

drawRegisterScreen(): Tương tự drawLoginScreen() nhưng dùng dữ liệu registerUsername, registerPassword và registerButtons.

drawMainMenu(): Duyệt mainMenuButtons và gọi btn.draw(g) cho từng nút, Play, Controls, Settings, Quit.

drawStartMenu(): Duyệt startButtons và gọi btn.draw(g) cho nút level và nút Back to Menu.

drawLevelLockedPopup(): Vẽ hộp thoại bo tròn màu đen với viền trắng, hiển thị text "Level is Locked!" và nút đóng popup. Được gọi sau drawStartMenu() và drawOverlay() để popup nổi lên trên nền tối.

drawLevelInfoPopup(): Vẽ hộp thoại bo tròn hiển thị số màn đang chọn và điểm cao màu vàng. Bên dưới có nút Play và Cancel.

drawPauseMenu(): Vẽ text "PAUSED" và duyệt pauseButtons để vẽ các nút Resume, Restart, Menu.

drawQuitConfirm(): Duyệt quitConfirmButtons vẽ hai nút Yes và No xác nhận thoát game.

drawGame(): Gọi drawMap() vẽ bản đồ, drawEntities() vẽ nhân vật, vẽ điểm số góc trên trái, gọi drawGamePlay() vẽ nút Pause và Settings. Nếu isGameOver = true thì vẽ overlay tối, chồng ảnh bg_gameover và gọi drawGameOverScreen(). Nếu isGameWon = true thì tương tự với bg_gamewon và drawGameWonScreen().

drawGameOverScreen(): Vẽ điểm số căn giữa màn hình bằng FontMetrics rồi duyệt gameOverButtons vẽ nút Replay và Menu.

drawGameWonScreen(): Tương tự drawGameOverScreen() nhưng duyệt gameWonButtons vẽ nút Next Level và Menu.

drawSoundStatus(): Vẽ ảnh sound_on hoặc sound_off lên màn hình Settings tùy theo trạng thái SoundManager.isMuted().

3.3.4. GameWindow – Khung cửa sổ JFrame

GameWindow kế thừa JFrame, chịu trách nhiệm tạo và cấu hình cửa sổ game. Lớp này toàn bộ cấu hình được thực hiện trong constructor.

- Bảng phương thức:

GameWindow
Methods
GameWindow(GamePanel gamePanel)

Hình 3. 31: Bảng các phương thức trong class GamWindow

GameWindow(GamePanel gamePanel): Nhận GamePanel và thêm vào cửa sổ bằng add(gamePanel). Thiết lập tiêu đề, đóng cửa sổ, gọi pack() để tự động tính kích thước cửa sổ vừa khít với GamePanel, setLocationRelativeTo(null) để căn giữa màn hình, setResizable(false) để người dùng không thể thay đổi kích thước cửa sổ, cuối cùng setVisible(true) để hiển thị cửa sổ.

3.4. Lớp Utils – Dịch vụ dùng chung

3.4.1 Class AssetManager

AssetManager là lớp tiện ích quản lý toàn bộ tài nguyên ảnh và bản đồ của game theo mẫu thiết kế Singleton.

- Bảng thuộc tính trong class AssetMansger:

AssetManager	
Attributes	
static AssetManager instance	HashMap<String, char[][]> mapCache
HashMap<String, BufferedImage> imageCache	

Hình 3. 32: Bảng các thuộc tính trong class AssetManager

instance: Biến static lưu instance duy nhất của AssetManager. Được khởi tạo lần đầu khi getInstance() được gọi, các lần sau trả về instance đã có.

imageCache: HashMap lưu các ảnh đã load với key là tên ảnh và value là BufferedImage. Mỗi ảnh chỉ đọc từ file một lần duy nhất, các lần sau lấy thẳng từ cache, tránh đọc file lặp lại mỗi frame gây giật lag.

mapCache: HashMap lưu các bản đồ đã load với key là tên file và value là char[][]. Tương tự imageCache, mỗi file bản đồ chỉ đọc một lần và cache lại để dùng nhiều lần.

- Bảng phương thức trong AssetManager:

AssetManager
Methods
static AssetManager getInstance() BufferedImage getImage(String imageName) char[][] getMap(String mapName) char[][] loadMapFromFile(String mapName)

Hình 3. 33: Bảng các phương thức trong class AssetManager

getInstance(): Kiểm tra nếu instance == null thì tạo mới, ngược lại trả về instance đã có.

getImage(String imageName): Kiểm tra imageCache, nếu chưa có thì đọc file /assets/imageName.png từ classpath qua getResourceAsStream(), đọc thành BufferedImage bằng ImageIO.read() và lưu vào cache. Nếu file không tồn tại thì in lỗi ra console và trả về null. Các lần gọi sau với cùng tên thì trả về thẳng từ cache.

getMap(String mapName): Kiểm tra mapCache, nếu chưa có thì gọi loadMapFromFile() để đọc file và lưu vào cache. Trả về char[][] đã cache.

loadMapFromFile(String mapName): Đọc file /maps/mapName.txt từ classpath, duyệt từng dòng bằng BufferedReader, bỏ qua dòng rỗng, lưu vào ArrayList<String>. Sau đó tạo ma trận char[rows][cols] và điền từng ký tự từ mỗi dòng vào đúng vị trí. Nếu dòng ngắn hơn số cột thì điền ‘ ’ cho các ô thiếu để tránh lỗi truy cập ngoài phạm vi.

3.4.2. Class SoundManager

SoundManager quản lý toàn bộ âm thanh trong game thông qua Java Sound API.

- Bảng thuộc tính của class SoundManager:

SoundManager	
Attributes	
static SoundManager instance	boolean isMuted
HashMap<String, Clip> soundCache	Set<String> loopingClips

Hình 3. 34: Bảng các thuộc tính trong class SoundManager

instance: Biến static lưu instance duy nhất, khởi tạo lần đầu khi getInstance() được gọi.

soundCache: HashMap lưu các Clip đã load với key là tên file âm thanh. Clip là đối tượng Java Sound API cho phép phát, dừng và tua lại âm thanh. Mỗi file chỉ load một lần và tái sử dụng nhiều lần.

isMuted: Biến bật/tắt tiếng toàn bộ game. Khi true, mọi lệnh playSound() và loopSound() đều bị bỏ qua.

loopingClips: Set lưu tên các âm thanh đang được lặp liên tục. Dùng để biết cần phát lại âm thanh nào khi người dùng bật tiếng trở lại sau khi tắt.

- Bảng phương thức của class SoundManager:

SoundManager	
Methods	
static SoundManager getInstance()	void stopAndRemoveLoop(String soundName)
void loadSound(String soundName)	void stopAllLooping()
void playSound(String soundName)	boolean isMuted()
void loopSound(String soundName)	void toggleMute()
void stopSound(String soundName)	

Hình 3. 35: Bảng các phương thức trong class SoundManager

loadSound(String soundName): Đọc file /sounds/soundName từ classpath, tạo AudioInputStream và mở vào Clip. Chỉ load nếu chưa có trong soundCache, tránh load lại file đã tồn tại. Được GameController gọi một lần duy nhất khi khởi động.

playSound(String soundName): Nếu đang tắt tiếng thì bỏ qua. Lấy Clip từ cache, nếu đang phát thì dừng lại (stop()), tua về đầu (setFramePosition(0)) rồi phát (start()).

loopSound(String soundName): Thêm tên âm thanh vào loopingClips trước, sau đó nếu không tắt tiếng thì tua về đầu và gọi clip.loop(Clip.LOOP_CONTINUOUSLY) để lặp vô hạn. Thêm vào loopingClips trước để nếu đang tắt tiếng, khi bật lại toggleMute() vẫn biết cần phát lại âm thanh này.

stopSound(String soundName): Dừng một âm thanh cụ thể nếu đang phát. Không xóa khỏi loopingClips.

stopAndRemoveLoop(String soundName): Dừng âm thanh và xóa khỏi loopingClips.

stopAllLooping(): Dừng toàn bộ âm thanh đang lặp và xóa sạch loopingClips. Được gọi khi Restart hoặc chuyển màn để đảm bảo không có âm thanh nào còn phát từ màn trước.

toggleMute(): Đảo giá trị isMuted. Nếu vừa tắt tiếng thì dừng tất cả Clip đang phát. Nếu vừa bật tiếng trở lại thì duyệt loopingClips và phát lại tất cả âm thanh đang cần lặp.

3.4.3. Class DatabaseManager

DatabaseManager quản lý toàn bộ tương tác với cơ sở dữ liệu MySQL thông qua JDBC, sử dụng XAMPP làm môi trường chạy MySQL local.

- Bảng thuộc tính của class DatabaseManager:

DatabaseManager	
Attributes	
static DatabaseManager instance	static final String PASS = ""
static final String DB_URL	Connection connection
static final String DB_NAME = "pacman_game"	
static final String USER = "root"	

Hình 3. 36: Bảng các thuộc tính trong class DatabaseManager

instance: Biến static lưu đối tượng duy nhất của DatabaseManager. Được khởi tạo lần đầu khi getInstance() được gọi.

DB_URL, DB_NAME, USER, PASS: Thông tin kết nối cố định đến MySQL chạy trên XAMPP, database pacman_game, tài khoản mặc định root không mật khẩu.

connection: Đối tượng Connection JDBC duy trì kết nối đến database trong suốt vòng đời ứng dụng. Được khởi tạo một lần trong initDatabase() và dùng chung cho tất cả các thao tác SQL.

- Bảng phương thức của class DatabaseManager:

DatabaseManager	
Methods	
static DatabaseManager getInstance()	int getUnlockedMaxLevel(String username)
void initDatabase()	int getHighScore(String username, int levelId)
String hashPassword(String password)	void updateScoreAndUnlockNext(String username, int currentLevelId, int score)
boolean register(String username, String password)	
boolean login(String username, String password)	

Hình 3. 37: Bảng các phương thức trong class DatabaseManager

getInstance(): Triển khai Singleton, tạo instance mới nếu chưa có, ngược lại trả về instance đã có. Constructor tự động gọi initDatabase() khi tạo lần đầu.

initDatabase(): Kết nối đến MySQL server, tạo database pacman_game nếu chưa tồn tại bằng "CREATE DATABASE IF NOT EXISTS". Sau đó kết nối lại vào đúng database và tạo hai bảng: users và user_progress.

hashPassword(String password): Mã hóa mật khẩu bằng thuật toán SHA-256 trước khi lưu vào database. Dùng MessageDigest để tạo mảng byte hash, sau đó chuyển sang chuỗi hex 64 ký tự.

register(String username, String password): Kiểm tra username và password không rỗng, hash mật khẩu rồi thực thi "INSERT INTO users". Nếu username đã tồn tại thì trả về false, vì username được khai báo "UNIQUE" trong bảng.

login(String username, String password): Hash mật khẩu nhập vào rồi so sánh với password_hash trong database bằng “SELECT”. Nếu ResultSet có dòng trả về thì đăng nhập thành công (rs.next() trả về true).

getUnlockedMaxLevel(String username): Truy vấn MAX(level_id) từ user_progress của người dùng để lấy màn cao nhất đã mở khóa. Nếu chưa có dữ liệu hoặc kết quả < 1 thì trả về 1, đảm bảo người dùng mới luôn có thể chơi màn 1.

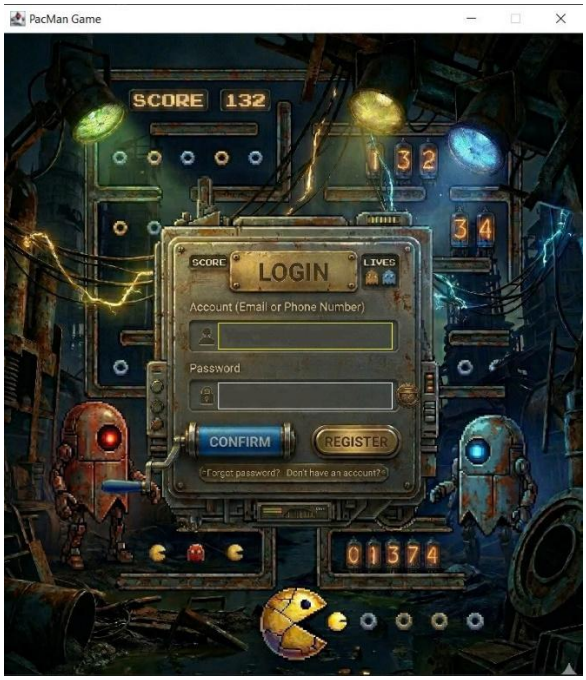
getHighScore(String username, int levelId): Truy vấn high_score của người dùng tại màn cụ thể. Trả về -1 nếu người dùng chưa từng chơi màn đó.

updateScoreAndUnlockNext(String username, int currentLevelId, int score): Thực hiện hai thao tác: đầu tiên dùng "INSERT INTO user_progress (user_id, level_id, high_score) VALUES (?, ?, ?) ON DUPLICATE KEY UPDATE high_score = ?" để lưu điểm cao của màn vừa hoàn thành, nếu chưa có thì thêm mới, nếu đã có thì cập nhật đè lên. Sau đó nếu currentLevelId < 9 thì dùng “INSERT IGNORE” để mở khóa màn tiếp theo, “INSERT IGNORE” bỏ qua nếu bản ghi đã tồn tại.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

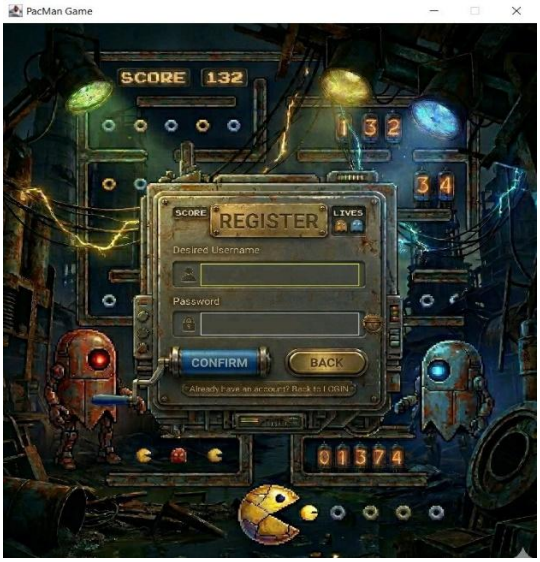
4.1. Kết quả đạt được

4.1.1. Demo màn hình Login / Register



Hình 4. 1: Màn hình Login

Màn hình đăng nhập cho phép người dùng nhập tên đăng nhập và mật khẩu để truy cập vào game Ô nhập liệu đang được chọn được tô viền vàng để người dùng dễ nhận biết. Mật khẩu có thể ẩn/hiện thông qua nút mắt ở góc phải. Hệ thống xác thực thông tin qua database và hiển thị thông báo lỗi nếu sai tài khoản.



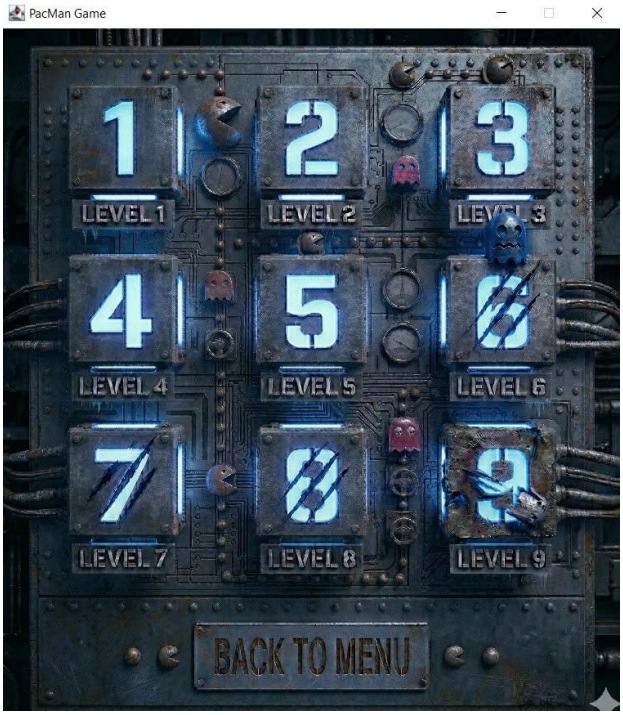
Hình 4. 2: Màn hình Register

Màn hình đăng ký có cấu trúc tương tự đăng nhập. Hệ thống kiểm tra độ dài tối thiểu 3 ký tự cho cả username và password trước khi tạo tài khoản.

4.1.2. Demo màn hình chọn màn chơi

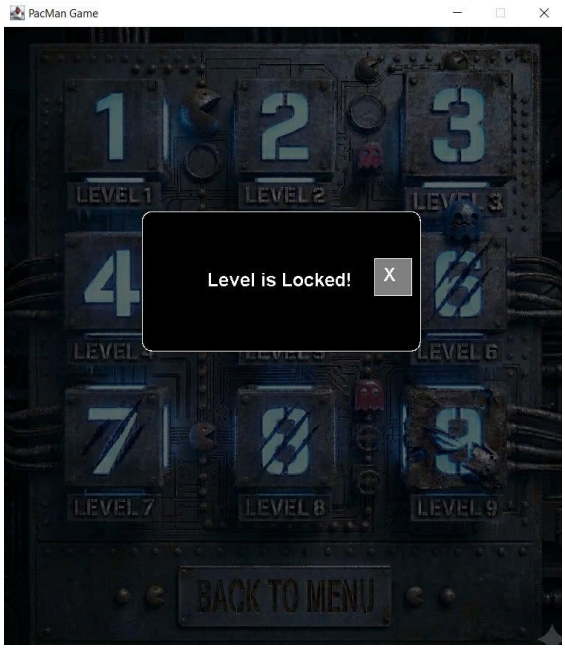
Màn hình chọn leve là một trong những giao diện quan trọng giúp người chơi dễ dàng điều hướng giữa các màn chơi. Giao diện này được thiết kế theo dạng lưới, lấy cảm hứng từ bảng điều khiển máy móc cũ, hiển thị rõ ràng tất cả các level có trong game dưới dạng các nút.

Ngoài ra, màn hình còn tích hợp nút "BACK TO MENU" ở phía dưới cùng, cho phép người chơi dễ dàng quay về màn hình chính bất kỳ lúc nào.



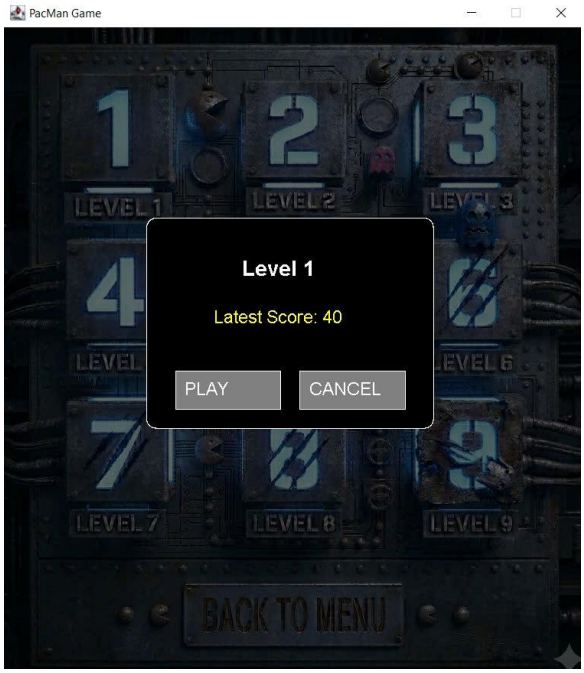
Hình 4. 3: Màn hình chọn level

Khi người chơi nhấn vào một nút level, hệ thống sẽ phản hồi linh hoạt tùy theo trạng thái của level đó. Với level chưa mở khóa, một popup thông báo sẽ xuất hiện để thông báo level đang bị khóa.



Hình 4. 4: Popup level khóa

Ngược lại, với level đã mở khóa, popup sẽ hiển thị thông tin chi tiết bao gồm điểm số cao nhất, số sao đạt được và các lựa chọn để bắt đầu chơi hoặc quay lại.



Hình 4. 5: Popup thông tin level

4.1.3. Demo màn hình Gameplay thực tế

Màn hình gameplay hiển thị bản đồ mê cung, PacMan, các con ghost và cherry. Điểm số hiện tại được cập nhật liên tục ở góc trên trái. PacMan di chuyển mượt mà với cơ chế hàng đợi hướng rẽ, người chơi có thể nhấn phím trước ngã rẽ và PacMan tự rẽ khi đến đúng vị trí. Bốn con ghost di chuyển độc lập với AI ngẫu nhiên, tự đổi hướng tại các ngã rẽ.

Hình 4. 6: Màn hình Gameplay

Màn hình tạm dừng được kích hoạt khi người chơi tương tác với nút pause trong lúc đang chơi game. Màn hình này cho phép người chơi tạm ngừng trò chơi, cân nhắc các lựa chọn tiếp theo mà không làm mất tiến độ hiện tại.

Hình 4. 7: Màn hình tạm dừng

4.1.4. Demo màn hình GameOver/GameWon

Khi PacMan bị ghost đụng, màn hình Game Over hiển thị với điểm số cuối cùng căn giữa. Người chơi có thể chọn chơi lại hoặc về menu chính.

Hình 4. 8: Màn hình GameOver

Khi ăn hết toàn bộ cherry trên bản đồ, màn hình Game Won hiển thị với điểm số cuối cùng. Người chơi có thể chọn chơi lại, về menu hoặc sang màn tiếp theo.

Hình 4. 9: Màn hình GameWon

4.2. Đánh giá ưu điểm

Dự án đã xây dựng thành công game Pac-Man hoàn chỉnh với đầy đủ các tính năng cơ bản.

Về kiến trúc, dự án áp dụng mô hình MVC một cách rõ ràng và nhất quán, GameModel chỉ lưu trữ dữ liệu, GameController xử lý toàn bộ logic, GamePanel chỉ đọc dữ liệu và hiển thị. Sự phân tách này giúp code dễ bảo trì, dễ mở rộng và hạn chế sự phụ thuộc giữa các thành phần.

Về hiệu năng, cơ chế delta time loop giúp game chạy ổn định đúng 60 FPS bất kể tốc độ xử lý của máy tính. Việc sử dụng AssetManager theo Singleton để cache ảnh và bản đồ giúp tránh đọc file lặp lại nhiều lần trong quá trình chơi, giảm thiểu thời gian tải và tăng hiệu năng đáng kể.

Về bảo mật, hệ thống quản lý tài khoản người dùng tích hợp database MySQL với mã hóa mật khẩu SHA-256 — mật khẩu gốc không bao giờ được lưu trực tiếp vào database, đảm bảo an toàn thông tin ngay cả khi database bị lộ.

Về trải nghiệm người dùng, cơ chế hàng đợi hướng rẽ giúp PacMan di chuyển mượt mà và phản hồi tốt với thao tác của người chơi. Hiệu ứng hover animation trên các nút bấm tạo giao diện chuyên nghiệp. Hệ thống quản lý âm thanh linh hoạt với khả năng phát, dừng, lặp và bật tắt tiếng không gây xung đột giữa các trạng thái game.

Về tính năng, game hỗ trợ nhiều màn chơi với cơ chế mở khóa dần theo tiến trình, lưu điểm cao vào database và hiển thị lại cho người chơi trước mỗi lượt chơi. Hệ thống chuyển cảnh được tổ chức tập trung qua handleButtonClick() giúp luồng hoạt động rõ ràng và dễ theo dõi.

4.3. Hạn chế và hướng phát triển

4.3.1. Hạn chế

Mặc dù đã đạt được các mục tiêu cơ bản, dự án vẫn còn một số hạn chế cần được cải thiện.

Về AI ghost, các con ma hiện tại chỉ di chuyển ngẫu nhiên tại các ngã rẽ, chưa có thuật toán truy đuổi PacMan. Sự thiếu hụt này làm giảm độ thách thức và tính chiến thuật của game.

Về gameplay, game chưa có hệ thống mạng sống, người chơi thua ngay lập tức khi bị ghost chạm lần đầu. Game cũng chưa có power pellet cho phép PacMan ăn ghost trong thời gian ngắn. Số lượng level còn hạn chế và các bản đồ chưa đủ đa dạng để tạo trải nghiệm phong phú cho người chơi.

Về kỹ thuật, kích thước bản đồ hiện đang được kiểm tra cứng với 19 cột và 21 hàng trong canMove(), thay vì đọc động từ Map. Điều này làm giảm tính linh hoạt khi muốn thêm bản đồ có kích thước khác.

4.3.2. Hướng phát triển

Trong tương lai, dự án có thể được phát triển theo nhiều hướng. Về AI, có thể nâng cấp thuật toán di chuyển của ghost hoặc tái hiện đúng hành vi của từng con ma

trong game gốc, tăng độ thách thức và tính chiến thuật. Về gameplay, có thể bổ sung hệ thống mạng sống, power pellet, các loại vật phẩm đặc biệt và chế độ chơi nhiều người. Ngoài ra, hệ thống có thể được mở rộng để hỗ trợ người dùng tự tạo bản đồ riêng thông qua một công cụ thiết kế level đơn giản.

KẾT LUẬN

Sau một học kỳ nghiên cứu và thực hiện, nhóm đã hoàn thành dự án xây dựng game Pac-Man hoàn chỉnh trên nền tảng Java. Từ một ý tưởng bắt đầu bằng tờ giấy trắng, nhóm đã dần hình thành một ứng dụng desktop có thể chơi được thực sự, tích hợp đầy đủ giao diện đồ họa (Java Swing/AWT), âm thanh (Java Sound API), hệ thống đăng nhập và lưu trữ tiến trình (MySQL/JDBC qua XAMPP), tất cả được tổ chức theo kiến trúc MVC mạch lạc.

Qua quá trình thực hiện dự án, nhóm đã nắm vững và áp dụng được nhiều kiến thức quan trọng trong lập trình Java. Về lập trình hướng đối tượng, dự án thể hiện rõ các nguyên tắc đóng gói, kế thừa và phân tách trách nhiệm thông qua việc tổ chức các class theo đúng vai trò trong kiến trúc MVC. Về lập trình game, nhóm đã hiểu và triển khai được vòng lặp game với delta time, cơ chế phát hiện va chạm AABB, hệ thống quản lý trạng thái và AI di chuyển ngẫu nhiên cho ghost. Về tích hợp hệ thống, nhóm đã kết nối thành công với cơ sở dữ liệu MySQL, xây dựng hệ thống đăng ký đăng nhập với mã hóa SHA-256 và lưu trữ tiến trình người chơi.

Nhìn lại hành trình, nhóm nhận thấy rằng giá trị lớn nhất không chỉ là sản phẩm cuối cùng, mà là quá trình đối mặt và giải quyết từng vấn đề thực tế: từ việc khiến PacMan rẽ mượt mà tại góc cua, đến việc đảm bảo ghost không bao giờ đi xuyên tường, hay thiết kế một hệ thống quản lý trạng thái không bị lẫn lộn khi có nhiều màn hình. Những hạn chế còn lại như AI ghost chưa đủ thông minh hay gameplay chưa phong phú hoàn toàn là những bài toán mở cho tương lai. Nhóm em xin chân thành cảm ơn cô Đào Thị Lệ Thủy đã đồng hành và hướng dẫn trong suốt học kỳ này.